



Security Audit

ADIL Chain (DeFi)

Table of Contents

Executive Summary	4
Project Context	4
Audit Scope	7
Security Rating	12
Intended Smart Contract Functions	13
Code Quality	15
Audit Resources	15
Dependencies	15
Severity Definitions	16
Status Definitions	17
Audit Findings	18
Centralisation	74
Conclusion	75
Our Methodology	76
Disclaimers	78
About Hashlock	79

CAUTION

THIS DOCUMENT IS A SECURITY AUDIT REPORT AND MAY CONTAIN CONFIDENTIAL INFORMATION. THIS INCLUDES IDENTIFIED VULNERABILITIES AND MALICIOUS CODE WHICH COULD BE USED TO COMPROMISE THE PROJECT. THIS DOCUMENT SHOULD ONLY BE FOR INTERNAL USE UNTIL ISSUES ARE RESOLVED. ONCE VULNERABILITIES ARE REMEDIATED, THIS REPORT CAN BE MADE PUBLIC. THE CONTENT OF THIS REPORT IS OWNED BY HASHLOCK PTY LTD FOR USE OF THE CLIENT.

Executive Summary

The ADIL Chain team partnered with Hashlock to conduct a security audit of their smart contracts. Hashlock manually and proactively reviewed the code in order to ensure the project's team and community that the deployed contracts are secure.

Project Context

AdilChain is a hybrid blockchain platform that combines decentralized architecture with selective administrative controls to enhance security and prevent fraud. It operates on a Proof-of-Stake consensus mechanism, designed for scalability, efficiency, and low transaction costs, while maintaining full compatibility with the Ethereum Virtual Machine (EVM) to allow seamless integration of Ethereum-based smart contracts. The platform also issues its own token standard and supports stable token swaps, positioning itself within the DeFi and Web3 ecosystem. The audit focuses on evaluating the smart contract implementation and overall system architecture to ensure security, accuracy, and compliance with the project's intended functionality.

Project Name: ADIL Chain

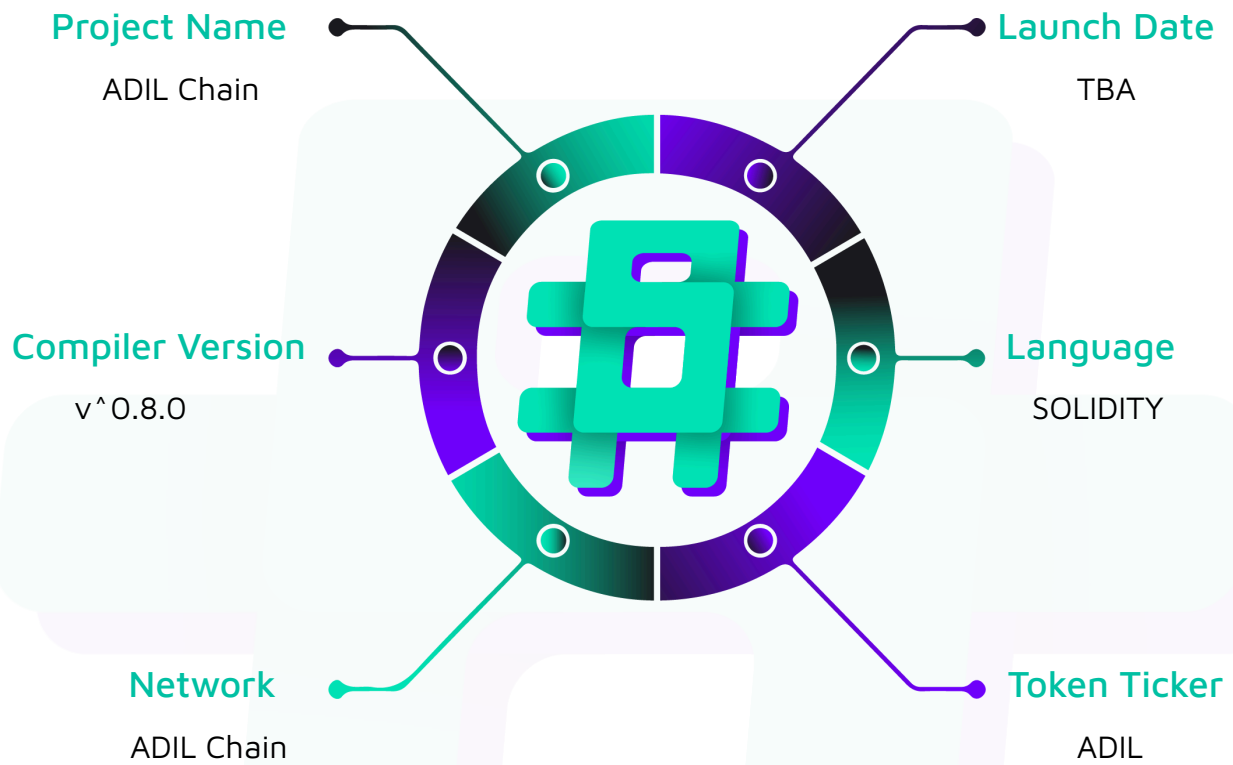
Project Type: DeFi

Compiler Version: ^0.8.0

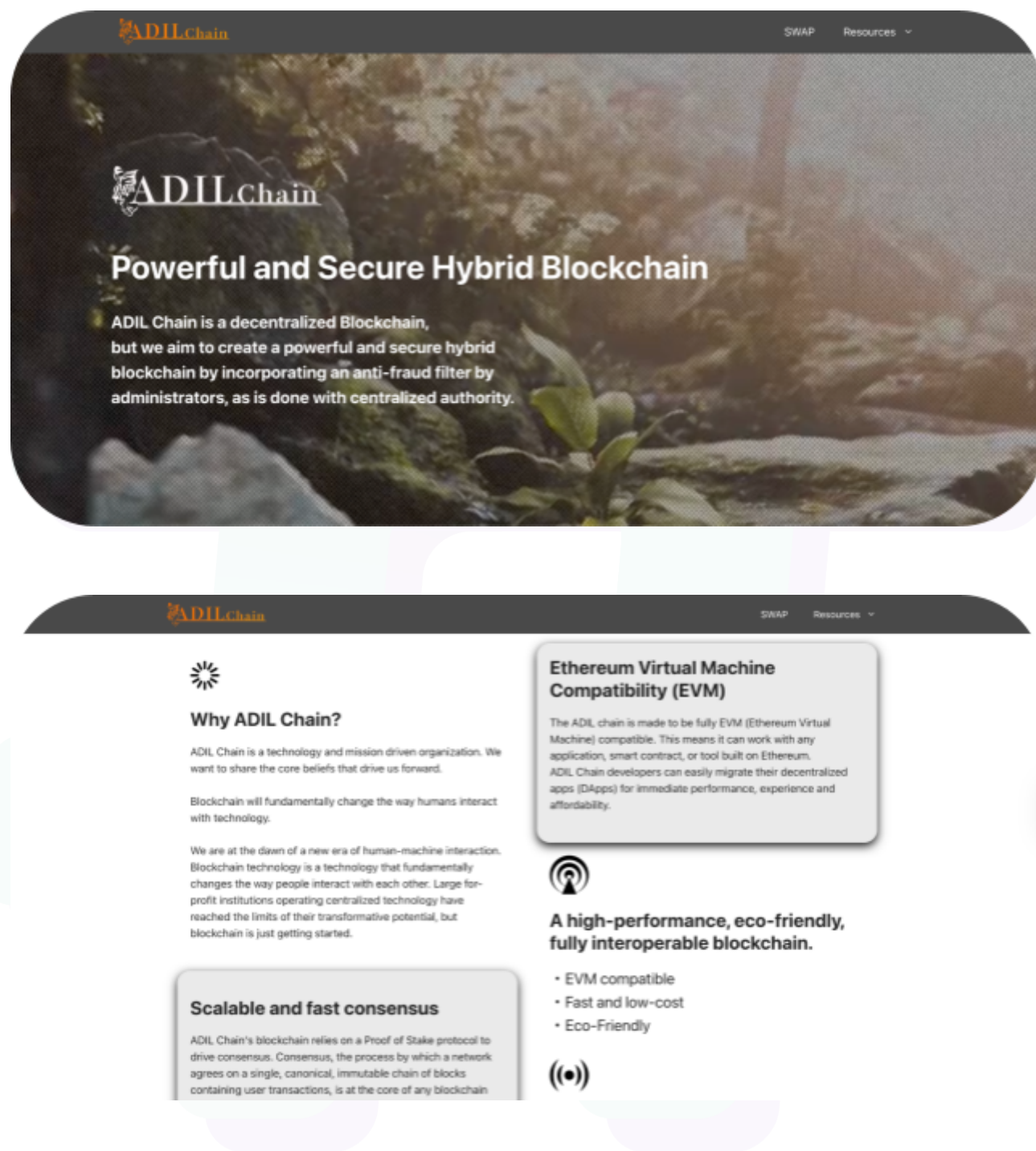
Website: <https://adilchain.io/>

Logo:



Visualised Context:

Project Visuals:



Audit Scope

We at Hashlock audited the solidity code within the ADIL Chain project. The scope of work included a comprehensive review of the smart contracts listed below. We tested the smart contracts to check for their security and efficiency. These tests were undertaken primarily through manual line-by-line analysis and were supported by software-assisted testing.

Description	Adil Chain Smart Contracts
Platform	ADIL Chain / Solidity
Audit Date	October, 2025
Contract 1	adil-chain-core-contracts/v1/Params.sol
Contract 2	adil-chain-core-contracts/v1/VotePool.sol
Contract 3	adil-chain-core-contracts/v1/Punish.sol
Contract 4	adil-chain-core-contracts/v1/library/SafeSend.sol
Contract 5	adil-chain-core-contracts/v1/library/SortedList.sol
Contract 6	adil-chain-core-contracts/v1/Validators.sol
Contract 7	adil-chain-core-contracts/v1/AddressList.sol
Contract 8	adil-chain-core-contracts/v1/interfaces/IPunish.sol
Contract 9	adil-chain-core-contracts/v1/Governance.sol
Contract 10	adil-chain-core-contracts/v1/library/ReentrancyGuard.sol
Contract 11	adil-chain-core-contracts/v1/interfaces/IValidators.so
Contract 12	adil-chain-core-contracts/v1/interfaces/IVotePool.sol

Contract 13	adil-chain-core-contracts/Params.sol
Contract 14	adil-chain-core-contracts/Punish.sol
Contract 15	adil-chain-core-contracts/Migrations.sol
Contract 16	adil-chain-core-contracts/library/SafeMath.sol
Contract 17	adil-chain-core-contracts/Validators.sol
Contract 18	adil-chain-core-contracts/Proposal.sol
Contract 19	tokenizing-contracts/libraries/OwnerOperatorAdmin.sol
Contract 20	tokenizing-contracts/libraries/OwnerOperator.sol
Contract 21	tokenizing-contracts/Tokenize.sol
Contract 23	tokenizing-contracts/TokenEMAS.sol
Contract 24	tokenizing-contracts/interfaces/ITokenMint.sol
Contract 25	adil-service-contracts/ADILServiceExchange.sol
Contract 26	adil-service-contracts/ADILServiceSubscriptionDistribution.sol
Contract 27	adil-service-contracts/ADILServicePayment.sol
Contract 28	adil-service-contracts/proxy/ProxyADILGameServiceExchange.sol
Contract 29	adil-service-contracts/proxy/pools/ProxyADILServiceSubscriptionDistribution.sol
Contract 30	adil-service-contracts/proxy/pools/hexa/ProxyHexaClaimPool.sol
Contract 31	adil-service-contracts/proxy/pools/hexa/ProxyHexaBurnPool.sol

Contract 32	adil-service-contracts/proxy/pools/alpha/ProxyAlphaClaimPool.sol
Contract 33	adil-service-contracts/proxy/pools/alpha/ProxyAlphaBurnPool.sol
Contract 34	adil-service-contracts/proxy/pools/ProxyADILServiceBonusDistribution.sol
Contract 35	adil-service-contracts/proxy/ProxyADILServiceTicketDistribution.sol
Contract 36	adil-service-contracts/proxy/ProxyADILServiceTicket.sol
Contract 37	adil-service-contracts/proxy/ProxyADILServicePharmacy.sol
Contract 38	adil-service-contracts/proxy/ProxyADILServicePayment.sol
Contract 39	adil-service-contracts/proxy/ProxyADILServiceExchange.sol
Contract 40	adil-service-contracts/proxy/ProxyADILGameServiceAdilVault.sol
Contract 41	adil-service-contracts/proxy/ProxyLifeTicketDistribution.sol
Contract 42	adil-service-contracts/proxy/ProxyADILGameServiceVault.sol
Contract 43	adil-service-contracts/ADILGameServiceVault.sol
Contract 44	adil-service-contracts/ADILServiceBonusClaimPool.sol
Contract 45	adil-service-contracts/admin/ProxyRoot.sol
Contract 46	adil-service-contracts/ADILServiceBonusDistribution.sol
Contract 47	adil-service-contracts/libraries/OwnableUpgradeable.sol
Contract 48	adil-service-contracts/libraries/OwnerOperator.sol
Contract 49	adil-service-contracts/libraries/TransferHelper.sol

Contract 50	adil-service-contracts/libraries/VerifySignature.sol
Contract 51	adil-service-contracts/libraries/BytesLibrary.sol
Contract 52	adil-service-contracts/common/ADILServiceCommon.sol
Contract 53	adil-service-contracts/ADILServiceBonusBurnPool.sol
Contract 54	adil-service-contracts/TokenADUSD.sol
Contract 55	adil-service-contracts/ADILServiceTicketDistribution.sol
Contract 56	adil-service-contracts/ADILGameServiceAdilVault.sol
Contract 57	adil-service-contracts/ADILServiceTicket.sol
Contract 58	adil-service-contracts/ADILServicePharmacy.sol
Contract 59	adil-service-contracts/ADILGameServiceExchange.sol
Contract 60	adil-service-contracts/interfaces/IDistribute.sol
Contract 61	adil-service-contracts/interfaces/ITicket.sol
Contract 62	adil-service-contracts/interfaces/IERC20Token.sol
Contract 63	adil-service-contracts/interfaces/IMSPPackageBuy.sol
Contract 64	adil-swap-contracts/ExchangeUpgradeable.sol
Contract 65	adil-swap-contracts/proxy/ProxyExchange.sol
Contract 66	adil-swap-contracts/proxy/ProxyRoot.sol
Contract 67	adil-swap-contracts/proxy/ProxyTronExchange.sol
Contract 68	adil-swap-contracts/Exchange.sol
Contract 69	adil-swap-contracts/libraries/OwnableUpgradeable.sol
Contract 70	adil-swap-contracts/libraries/OwnerOperator.sol

Contract 71	adil-swap-contracts/libraries/VerifySignature.sol
Contract 72	adil-swap-contracts/libraries/TronVerifySignature.sol
Contract 73	adil-swap-contracts/TronExchangeUpgradeable.sol
Contract 74	adil-swap-contracts/interfaces/IERC20.sol
Audited GitLab Commit Hash	51751be2d2bbc30196368ad8a29c2abdc874e995
Fix Review GitLab Commit Hash	b70ff737a7439823d57b9076596a78fcd09850e7

Security Rating

After Hashlock's Audit, we found the smart contracts to be **"Secure"**. The contracts all follow simple logic, with correct and detailed ordering. They use a series of interfaces, and the protocol uses a list of Open Zeppelin contracts.



Not Secure

Vulnerable

Secure

Hashlocked

The 'Hashlocked' rating is reserved for projects that ensure ongoing security via bug bounty programs or on-chain monitoring technology.

All issues uncovered during automated and manual analysis were meticulously reviewed and applicable vulnerabilities are presented in the [Audit Findings](#) section. The list of audited assets is presented in the [Audit Scope](#) section, and the project's contract functionality is presented in the [Intended Smart Contract Functions](#) section.

All vulnerabilities initially identified have now been resolved or acknowledged.

Hashlock found:

3 High severity vulnerabilities

17 Medium severity vulnerabilities

4 Low severity vulnerabilities

Caution: Hashlock's audits do not guarantee a project's success or ethics, and are not liable or responsible for security. Always conduct independent research about any project before interacting.

Intended Smart Contract Functions

Claimed Behaviour	Actual Behaviour
adil-service-contracts - ERC-4626 tokenized vault that time-locks all user deposits. - Provides various on-chain services like payments, pharmacy orders, and gaming vaults, authorized via off-chain signatures. - The owner can sweep all underlying assets to a designated multisig wallet at any time. - Allows a designated owner/multisig to sweep underlying assets from the contracts. - Ensures secure management and transfer of user funds for specified services.	Contracts achieve this functionality.
adil-core-contracts - Provides the foundational on-chain logic for a Proof-of-Authority (PoA) or Proof-of-Stake (PoS) style consensus mechanism. - Manages the lifecycle of network validators, including their registration, removal, and punishment for misbehavior. - Implements a governance system allowing participants to create and vote on proposals for protocol changes. - Defines and manages critical network-wide parameters through a Params contract.	Contracts achieve this functionality.
adil-swap-contracts - Functions as a decentralized exchange (DEX)	Contracts achieve this functionality.

implementing an Automated Market Maker (AMM) model. - Facilitates the trustless swapping of various ERC20 tokens. - Allows users to act as Liquidity Providers (LPs) by depositing token pairs into liquidity pools. - Enables LPs to earn trading fees proportional to their share of the pool. - Maintains core AMM economic invariants (e.g., $x*y=k$) to determine token prices.	
tokenizing-contracts - TokenEMAS.sol : Implements a standard ERC20 token named "EMAS" with a fixed supply cap of 5 billion. - It includes Pausable functionality, allowing an owner to halt all token transfers. - It features an OwnerOperator access control model, where both an owner and a designated operator can mint new tokens up to the cap. - Any user can burn their own tokens. - Tokenize.sol : A contract that facilitates the swapping of different tokens against a base asset (ADIL). - It allows an operator to set and manage token prices relative to ADIL. - It supports configurable swap fees, which can be either a percentage or a fixed amount. - It maintains a list of valid swap pairs (from and to tokens). - It emits a Swap event upon a successful token exchange, logging the details of the transaction.	Contracts achieve this functionality.

Code Quality

This audit scope involves the smart contracts of the Adil Chain project, as outlined in the Audit Scope section. All contracts, libraries, and interfaces mostly follow standard best practices and to help avoid unnecessary complexity that increases the likelihood of exploitation, however, some refactoring were recommended to optimize security measures.

The code is very well commented on and closely follows best practice nat-spec styling. All comments are correctly aligned with code functionality.

Audit Resources

We were given the Adil Chain project smart contract code in the form of GitLab access.

As mentioned above, code parts are well commented. The logic is straightforward, and therefore it is easy to quickly comprehend the programming flow as well as the complex code logic. The comments are helpful in providing an understanding of the protocol's overall architecture.

Dependencies

As per our observation, the libraries used in this smart contracts infrastructure are based on well-known industry standard open source projects.

Apart from libraries, its functions are used in external smart contract calls.

Severity Definitions

The severity levels assigned to findings represent a comprehensive evaluation of both their potential impact and the likelihood of occurrence within the system. These categorizations are established based on Hashlock's professional standards and expertise, incorporating both industry best practices and our discretion as security auditors. This ensures a tailored assessment that reflects the specific context and risk profile of each finding.

Significance	Description
High	High-severity vulnerabilities can result in loss of funds, asset loss, access denial, and other critical issues that will result in the direct loss of funds and control by the owners and community.
Medium	Medium-level difficulties should be solved before deployment, but won't result in loss of funds.
Low	Low-level vulnerabilities are areas that lack best practices that may cause small complications in the future.
Gas	Gas Optimisations, issues, and inefficiencies.
QA	Quality Assurance (QA) findings are informational and don't impact functionality. Supports clients improve the clarity, maintainability, or overall structure of the code.

Status Definitions

Each identified security finding is assigned a status that reflects its current stage of remediation or acknowledgment. The status provides clarity on the handling of the issue and ensures transparency in the auditing process. The statuses are as follows:

Significance	Description
Resolved	The identified vulnerability has been fully mitigated either through the implementation of the recommended solution proposed by Hashlock or through an alternative client-provided solution that demonstrably addresses the issue.
Acknowledged	The client has formally recognized the vulnerability but has chosen not to address it due to the high cost or complexity of remediation. This status is acceptable for medium and low-severity findings after internal review and agreement. However, all high-severity findings must be resolved without exception.
Unresolved	The finding remains neither remediated nor formally acknowledged by the client, leaving the vulnerability unaddressed.

Audit Findings

High

[H-01] Exchange#depositExchange - Missing Native Token Validation Allows Fund Drainage

Description

The `depositExchange` function accepts native tokens through the `payable` modifier but lacks proper validation and handling when `tokenIn` represents the native token.

Vulnerability Details

The `depositExchange` function at line 290 is marked as `payable`, allowing it to receive native tokens (ETH, BNB, MATIC, etc.) via `msg.value`. However, the function's logic only handles ERC20 token transfers and completely ignores native token scenarios. At lines 329-336, the code checks if `tokenIn != address(0x0)` and processes it as an ERC20 token transfer, but there is no corresponding `else` block to handle the native token case where `tokenIn == address(0x0)`.

The critical vulnerability manifests in multiple ways: First, when a user intends to swap native tokens, they send `msg.value` along with their transaction, but the function never validates that `msg.value == amountIn`. A malicious user could specify `amountIn` as 1000 ETH while only sending 0.001 ETH via `msg.value`. The signature verification at lines 291-304 includes `amountIn` as a parameter, but this only validates that the off-chain signer approved that specific amount - it doesn't validate that the user actually sent that amount. Second, the native tokens sent via `msg.value` are not transferred to the `adminReceiveraddress` (unlike ERC20 tokens which go to `adminReceiver` at line 335). Instead, they remain stuck in the Exchange contract. Third, on the destination chain in the `withdrawExchange` function (lines 433-439), the contract distributes `amountOutof` native tokens to the user. Since users can claim arbitrary amounts without actually

depositing equivalent value, they could drain all native token liquidity from the destination chain contract.

The attack scenario is straightforward: An attacker calls `depositExchange` with `tokenIn = address(0)`, specifies a large `amountIn` (e.g., 1000 ETH), but only sends minimal `msg.value` (e.g., 0.001 ETH or even 0). The off-chain signer, unaware of the actual deposited amount, might approve the signature based on the declared amount. The transaction succeeds, creating a cross-chain swap record claiming 1000 ETH was deposited. On the destination chain, the attacker calls `withdrawExchange` (or waits for the operator to call it), and receives the full 1000 ETH equivalent in the destination token, despite only depositing 0.001 ETH. This can be repeated to drain the entire liquidity pool.

Impact

This vulnerability represents a complete breakdown of the cross-chain exchange mechanism for native tokens. Attackers can drain all native token funds locked in destination chain contracts by claiming to have deposited large amounts while actually depositing minimal or zero funds. The cross-chain bridge liquidity pools would be completely compromised, leading to total loss of funds held in the contracts. Legitimate users who have properly deposited funds would be unable to withdraw their assets once the pools are drained. The economic damage could be catastrophic, potentially resulting in millions of dollars in losses depending on the liquidity held in the contracts. This vulnerability makes the entire cross-chain exchange system unsafe for any native token operations and requires immediate remediation before any production deployment involving native tokens.

Recommendation

Implement comprehensive native token validation and handling logic. Add explicit checks to distinguish between native token and ERC20 token swaps. When `tokenIn` represents a native token (`address(0)` or a designated `NATIVE_TOKEN` constant), enforce strict validation that `msg.value == amountIn`. For ERC20 token swaps, enforce that `msg.value == 0` to prevent users from accidentally sending native tokens. The corrected logic should include: First, define a constant for native token representation

(e.g., `address constant NATIVE_TOKEN = address(0)`). Second, add validation immediately after signature verification: `if (tokenIn == NATIVE_TOKEN) { require(msg.value == amountIn, "Incorrect native token amount"); } else { require(msg.value == 0, "No ETH should be sent for ERC20 swaps"); IERC20(tokenIn).transferFrom(msg.sender, adminReceiver, amountIn); }`. Third, ensure native tokens received are properly forwarded to adminReceiver using `(bool success, ) = adminReceiver.call{value: msg.value}(""); require(success, "Native token transfer failed");`. Fourth, update the off-chain signing service to verify the actual transaction value matches the claimed amount before signature generation. This comprehensive validation ensures that users cannot claim to have deposited amounts that differ from their actual deposits.

Status

Resolved

[H-02] Validators.sol - Jailed Validators can also pass a proposal and bypass the jailing

Description

The system uses membership in `currentValidatorSet` to decide if an address is an active validator for governance, without filtering out `Status.Jailed`

Vulnerability Details

- `Validators.isActiveValidator(address)` returns `true` if who exists in `currentValidatorSet`, regardless of status.
- Proposal gates voting with `onlyValidator`, which calls `validators.isActiveValidator(msg.sender)`.
- Quorum is computed with `validators.getActiveValidators().length`, which returns the raw `currentValidatorSet` (unfiltered), so jailed validators inflate the denominator.
- When `slashed/removed`, `tryJailValidator` sets status to `Jailed` and removes from `highestValidatorsSet` only. The address often remains in `currentValidatorSet`, so it still passes `isActiveValidator` and is counted in quorum.

Slashing/jailing should remove or suspend governance rights. Here, jailed validators retain voting eligibility and quorum weight.

Impact

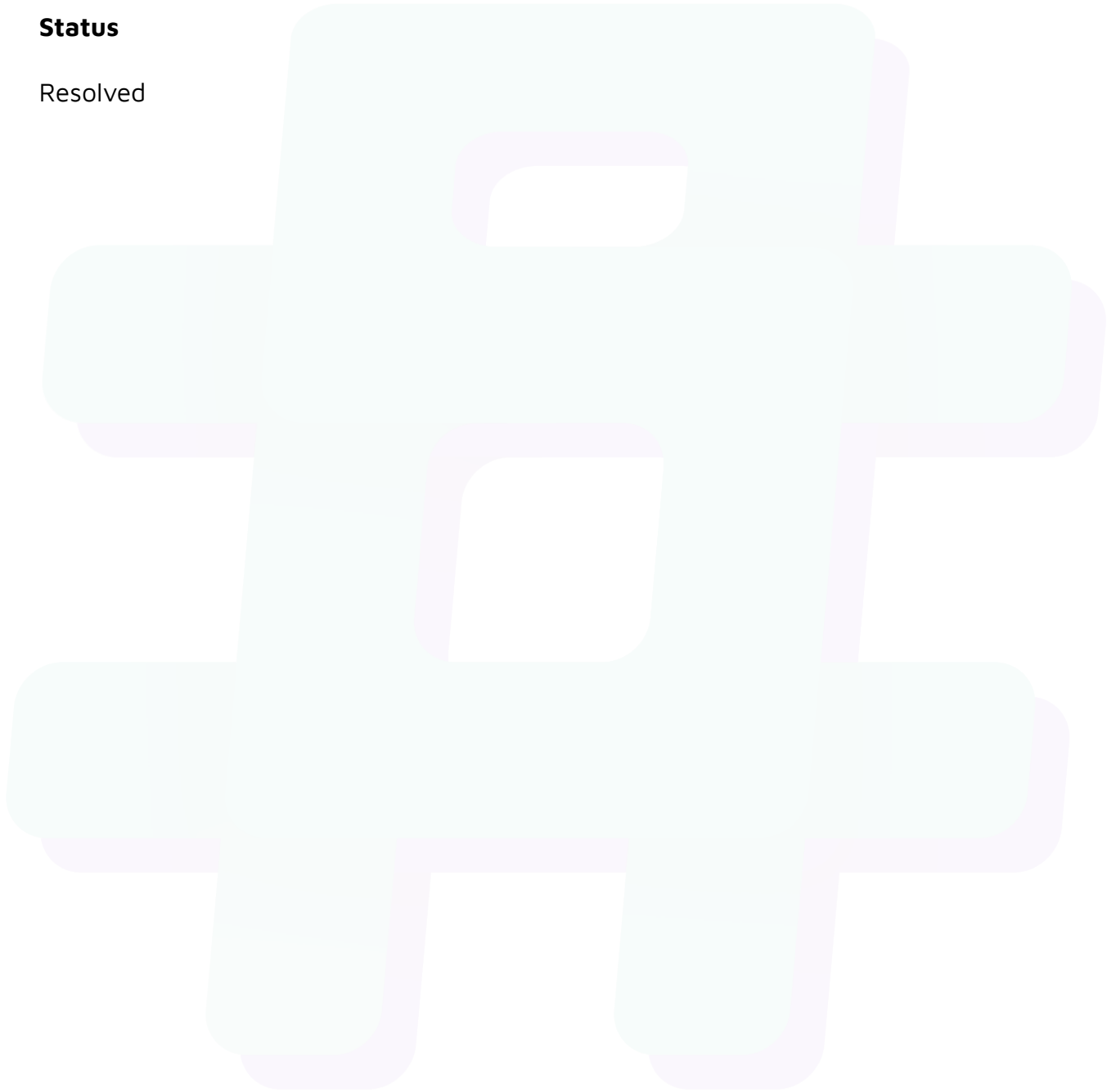
- Punished validators keep voting rights, undermining incentives and punishment.
- Quorum becomes harder to reach if many validators are jailed, potentially blocking upgrades.
- Attackers can intentionally get jailed to hinder governance while keeping seats in the quorum denominator.
- Validators in `Created/Unstaked` status still receive rewards if they remain in the current validator set. Only `Jailed` is excluded. This pays validators that are not fully active or below the minimum stake. Only the actively staked should get the rewards.

Recommendation

Exclude jailed validators from governance access control and quorum math. Punishment must remove power; otherwise jailing/slashing has weak deterrence.

Status

Resolved



[H-03] Tokenize#setADILPrice - Division by Zero Vulnerability in setADILPrice Function

Description

The `setADILPrice` function contains a conditional check for `price > 0` but allows the function to proceed with `price = 0`, which leads to a division by zero error in the price calculation formula.

Vulnerability Details

At line 223-226 in `Tokenize.sol:223`, the function checks if the price is greater than zero before calculating the `newPrice`. However, this check is implemented using a simple `if` statement rather than a `require` statement, which means the function will still execute when price is 0. When price equals 0, the formula at line 224 attempts to divide `PRICE_PRECISION.mul(PRICE_PRECISION)` by price, resulting in a division by zero error that will cause the transaction to revert. The calculation formula is: `newPrice = PRICE_PRECISION.mul(PRICE_PRECISION).div(price)` which translates to `newPrice = (1e9 * 1e9) / price` or `1e18 / price`. When price is 0, this operation will fail. Additionally, even if the division didn't fail, allowing a zero price would mean the ADIL token has no value, which would completely break the tokenomics of the system. The root cause is the use of an `if` statement instead of a `require` statement, which allows the function to set `adilPrice = price` even when price is 0, and then attempt the division operation that will inevitably fail.

Impact

This vulnerability has severe consequences for the protocol's operation and security. When an operator (who may be compromised or make an error) attempts to set the ADIL price to 0, the transaction will revert due to division by zero, preventing the price from being updated at all. While the revert protects against setting a zero price, it creates operational issues as the price update mechanism becomes unusable in error conditions. More critically, if the division operation somehow succeeded (in a different EVM implementation or future upgrade), it would allow the ADIL price to be set to 0, which would completely destroy the token's value and break all swap calculations throughout the protocol. This would make all tokens worthless as their prices are

derived from the ADIL price. The lack of proper input validation also indicates poor security practices that could lead to other unexpected behaviors. Users attempting to swap tokens when this error occurs would face transaction failures without clear error messages, leading to confusion and potential loss of gas fees.

Recommendation

The conditional check at line 223 should be converted from an if statement to a require statement to ensure the function reverts with a clear error message if an invalid price is provided. The code should be modified as follows: `require(price > 0, "ADIL price must be greater than zero");` and then proceed with the calculation `newPrice = PRICE_PRECISION.mul(PRICE_PRECISION).div(price);` without any conditional wrapper. This change ensures that any attempt to set a zero or negative price will be rejected immediately with a descriptive error message, preventing the division by zero error and protecting the protocol from invalid price updates. Additionally, consider implementing a reasonable minimum and maximum price range check to prevent extremely high or low prices that could cause calculation overflows or underflows. For example: `require(price >= MIN_PRICE && price <= MAX_PRICE, "Price out of acceptable range");` where `MIN_PRICE` and `MAX_PRICE` are predefined constants that represent reasonable bounds for the ADIL token price.

Status

Resolved

Medium

[M-01] VerifySignature - Missing Zero Address Validation in Signature Recovery

Description

The `ecrecover` function returns `address(0)` when signature verification fails, but this is not checked, potentially allowing invalid signatures to pass validation.

Vulnerability Details

The `recoverSigner` function at line 39-46 uses `ecrecover` to recover the signer's address from a signature. However, `ecrecover` is a low-level precompile that returns `address(0)` when the signature is invalid or malformed, rather than reverting. The contract does not check for this case before returning the recovered address. If a signature is crafted incorrectly (malformed `r`, `s`, `v` values) or if any error occurs during recovery, the function returns `address(0)`.

In the `verify` function at line 29-37, the recovered address is compared directly with the expected signer address using `recoverSigner(ethSignedMessageHash, signature) == _signer`. If the `signerAddress` is accidentally or maliciously set to `address(0)` in any of the consuming contracts (`ADILGameServiceAdilVault`, `ADILServicePayment`, `ADILServicePharmacy`, etc.), then any invalid or malformed signature would pass validation. While setting `signerAddress` to zero would require an error or malicious action by the owner/operator, the lack of validation creates a potential attack vector. Additionally, if there are any edge cases in signature handling that cause `ecrecover` to fail, the silent failure could lead to authorization bypasses.

Impact

If the signer address is set to `address(0)` (either through misconfiguration, initialization error, or malicious action), all signature validations would pass regardless of the actual signature provided. This would allow attackers to call any signature-protected function without proper authorization. Functions protected by signature verification would become open to exploitation, including critical operations like withdrawals, payments, order creation, and token claims. While this requires the `signerAddress` to be set to zero,



the lack of defensive programming creates unnecessary risk. The combination of this vulnerability with any initialization bugs or ownership compromise could lead to complete protocol failure and loss of all funds.

Recommendation

Add explicit validation in the `recoverSigner` function to check for the zero address and revert if `ecrecover` fails. Modify the function to include: `address signer = ecrecover(_ethSignedMessageHash, v, r, s); require(signer != address(0), "VerifySignature: Invalid signature"); return signer;` Additionally, add validation in all setter functions for `signerAddress` across all contracts to ensure it cannot be set to `address(0)`: `require(_signer != address(0), "Invalid signer address")`. Most contracts already have this check, but it should be consistently enforced. Consider adding an additional sanity check in the `verify` function itself to ensure the signer address is not zero before performing comparison.

Status

Acknowledged

[M-02] Exchange#constructor&setSignerAddress - Critical Signature Bypass via Zero Address Signer

Description

The contract lacks zero address validation for the signer address, which combined with ecrecover's behavior of returning `address(0)` for invalid signatures, creates a critical authentication bypass vulnerability.

[Extension of M-01]

Vulnerability Details

The signature verification system in this protocol has a fundamental flaw related to `address(0)` handling. The vulnerability exists at three critical points: First, in the constructor at line 83, the `signerAddress` is set without validating that `_signerAddress != address(0)`. Second, the `setSignerAddress` function at line 121 allows operators to change the signer address without zero address validation. Third, the `verifySignature` function in `VerifySignature.sol` at line 64 uses `ecrecover` (called within `recoverSigner`) which has a well-documented behavior of returning `address(0)` when signature recovery fails due to invalid signatures or incorrect parameters.

The Solidity `ecrecover` precompile returns `address(0)` in several scenarios: when the signature is malformed, when the `v` parameter is invalid (not 27 or 28), when the `s` or `r` values are outside the valid curve range, or when the signature doesn't correspond to any valid private key. The verification logic at line 64 simply compares `recoverSigner(ethSignedMessageHash, _signature) == _signer` without checking if the recovered address is `address(0)`. If `signerAddress` is set to `address(0)` (either through constructor initialization error, a malicious operator action, or an initialization bug), then any invalid or malformed signature would cause `ecrecover` to return `address(0)`, which would match the `signerAddress`, causing the verification to pass.

The attack scenarios include: First, if during deployment, the constructor is accidentally called with `address(0)` as the signer address (copy-paste error, wrong deployment script, etc.), all signature verifications would pass for any invalid signature. Second, a compromised or malicious operator could call `setSignerAddress(address(0))`, instantly compromising the entire signature verification system. Third, an attacker could craft

malformed signatures that cause `ecrecover` to return `address(0)` and bypass authentication if the signer address is zero. The impact is amplified because signature verification protects critical functions like `depositExchange` which handles user funds and cross-chain transfers.

Impact

If the signer address is set to `address(0)`, the entire signature verification mechanism becomes completely ineffective. Any user could call `depositExchange` with arbitrary parameters and completely invalid signatures, and all transactions would be accepted as valid. Attackers could create fraudulent cross-chain swap requests with any amounts, any tokens, and any exchange rates they choose. This would lead to immediate and complete compromise of the exchange system, allowing unlimited unauthorized fund movements. While setting the signer to zero requires either a deployment error or operator compromise, the lack of defensive programming creates unnecessary catastrophic risk. The combination of no zero address validation and no `ecrecover` return value checking represents a critical security vulnerability that violates basic smart contract security principles. Even if the likelihood of accidentally setting the signer to zero is low, the impact is so severe that this must be prevented at the contract level.

Recommendation

Implement multi-layered zero address validation to prevent this vulnerability. First, add validation in the constructor immediately after receiving the signer address parameter: `require(_signerAddress != address(0), "Exchange: Invalid signer address")`. Second, add the same validation in the `setSignerAddress` function at line 121: `require(_address != address(0), "Exchange: Invalid signer address")`. Third, improve the signature verification logic in `VerifySignature.sol` by adding explicit zero address checking. Modify the `verifySignature` function to: `address recoveredSigner = recoverSigner(ethSignedMessageHash, _signature); require(recoveredSigner != address(0), "Invalid signature"); return recoveredSigner == _signer;`. This three-layer defense ensures that: the signer address can never be set to zero during deployment or updates, signature verification explicitly rejects invalid signatures that cause `ecrecover` to return zero, and the system fails safely even if unexpected

conditions occur. Additionally, add similar validations for `adminReceiver` and `adminTransfer` addresses to maintain consistency and prevent configuration errors.

Status

Acknowledged

[M-03] ADILServiceExchange - Insecure Randomness in NFT Distribution

Description

The contract uses predictable blockchain variables (`block.timestamp`, `block.difficulty`) to generate random numbers for NFT selection.

Vulnerability Details

In the `swapLifeTicketForHorseNFT` function at line 70, the contract generates a random seed using `keccak256(abi.encodePacked(block.timestamp, msg.sender, block.difficulty, _ticketAmount))`. This seed is then used to select which NFTs to transfer to the user. All of these values are either publicly known or can be influenced by miners. The `block.timestamp` can be manipulated by miners within a certain range (approximately 15 seconds), `block.difficulty` is a public value known before the transaction is executed, and `msg.sender` and `_ticketAmount` are controlled by the user calling the function.

A malicious actor (especially a miner or validator) can predict the outcome of the randomness generation before submitting their transaction. They can calculate exactly which NFTs they would receive and only proceed with the transaction if they would get valuable NFTs. Regular users can also run simulations off-chain to determine the optimal time to call the function to get desired NFTs. The vulnerability is particularly severe because the random index is calculated as `uint256(keccak256(abi.encodePacked(randomSeed, i))) % balance` in a loop, making the selection pattern entirely deterministic once the initial seed is known. Furthermore, after the merge, `block.difficulty` is deprecated and always returns 0, making the randomness even more predictable.

Impact

This vulnerability allows sophisticated users and miners to manipulate the NFT distribution system to their advantage, receiving the most valuable NFTs while leaving less desirable ones for regular users. This creates an unfair distribution mechanism where those with technical knowledge or mining power can game the system. The rarity distribution of NFTs becomes meaningless as valuable NFTs will be cherry-picked

by attackers. This undermines the integrity of the entire NFT exchange system and can lead to loss of user trust, devaluation of the NFT collection, and economic losses for honest users who receive less valuable NFTs. The fairness principle of random distribution is completely violated.

Recommendation

Implement a secure randomness solution using Chainlink VR (Verifiable Random Function) or a similar oracle-based randomness provider. The swap process should be modified to a two-step commit-reveal scheme: first, the user commits to a swap by burning their tickets and locking in their request; second, after randomness is provided by the oracle, the NFTs are assigned and transferred. Example implementation: integrate Chainlink VRF by inheriting from `VRFConsumerBase`, request randomness when the user initiates a swap, and fulfill the NFT transfer in the callback function once secure randomness is received. This ensures that neither users nor miners can predict or manipulate the outcome of the NFT selection. If oracle solutions are not feasible, consider implementing a commit-reveal scheme where users submit a commitment hash in one transaction and reveal the nonce in a later transaction, with multiple blocks in between.

Status

Acknowledged

[M-04] ADILServiceBonusDistribution&ADILServiceSubscriptionDistribution - Incorrect Balance Check in Distribution Functions

Description

Distribution functions check `msg.sender` balance instead of contract balance when transferring tokens.

Vulnerability Details

In both `ADILServiceBonusDistribution` and `ADILServiceSubscriptionDistribution` contracts, the internal `transferERC20` function checks `erc20Token.balanceOf(msg.sender) >= _amount` before executing `erc20Token.transferFrom(msg.sender, _to, _amount)`. This logic is incorrect because the functions are called by operators/owners to distribute funds that should already be in the contract, not to transfer funds from the operator's personal wallet.

Looking at the flow: funds are meant to be sent to these distribution contracts first (from the payment contract using `TransferHelper.safeApprove` and then `distributeFunds`), and then the distribution contracts should distribute those funds to various pools. However, the `transferERC20` function is checking and transferring from `msg.sender` (the operator) instead of from the contract itself (`address(this)`). This means that for the distribution to work, the operator must personally hold all the tokens being distributed and approve them to the contract, which is not the intended design.

In `ADILServicePayment`, we can see that tokens are transferred to the payment contract first (`TransferHelper.safeTransferFrom(_token, msg.sender, address(this), _amount)` at line 40), then the payment contract approves tokens to the distribution contract (`TransferHelper.safeApprove(_token, ADILServiceDistribureAddress, _amount)` at line 42), and finally calls `ADILServiceDistribute.distributeFunds(alphaTypeId, _amount)`. The distribution contract should then transfer from its own balance, not from `msg.sender`.

Impact

The current implementation requires operators to personally hold and approve tokens for every distribution, which is operationally inefficient and error-prone. If the operator doesn't have sufficient balance or hasn't provided approval, the distribution will fail even though the distribution contract itself holds the funds. This breaks the intended fund flow and makes the distribution mechanism unreliable. The contract's token balance would accumulate over time without being properly distributed, while operators would need to constantly manage their personal token balances and approvals. This creates unnecessary operational overhead, increases the risk of distribution failures, and defeats the purpose of having separate distribution contracts. The system cannot function as designed because the distribution logic is fundamentally flawed.

Recommendation

Modify the `transferERC20` function in both contracts to transfer from the contract's own balance instead of from `msg.sender`. Replace the balance check with `require(erc20Token.balanceOf(address(this)) >= _amount, "Insufficient balance")` and change the transfer method from `erc20Token.transferFrom(msg.sender, _to, _amount)` to `erc20Token.transfer(_to, _amount)`. This aligns with the intended design where funds are first sent to the distribution contract, and then the contract distributes them to pools.

Alternatively, consider using OpenZeppelin's `SafeERC20` library which is already imported and provides safer transfer methods: `erc20Token.safeTransfer(_to, _amount)`.

Status

Acknowledged

[M-05] Exchange - Account Abstraction Wallet Incompatibility

Description

The contract hardcodes `msg.sender` as the recipient address, causing issues for users with Account Abstraction wallets where addresses differ across chains.

Vulnerability Details

In the `depositExchange` function at line 364, the contract stores `msg.sender` as the recipient address in the transaction info: `txnId, msg.sender, tokenIn, ...`. This address is used later in `withdrawExchange` on the destination chain to send tokens to the user. The fundamental problem is that with Account Abstraction (ERC-4337) wallets and some cross-chain wallet systems, a user's address on Chain A may be different from their address on Chain B, even though both are controlled by the same user.

Account Abstraction wallets work by deploying smart contract wallets that are controlled by the user. The deployment address of a smart contract depends on factors like the deployer address, nonce, and creation code. If a user has an AA wallet on Ethereum with address `0xABC...`, their corresponding AA wallet on Polygon might have address `0xDEF...` because the deployment conditions differed. When such a user tries to use the cross-chain exchange, they deposit from `0xABC...` on Ethereum, but the system tries to send tokens to `0xABC...` on Polygon where the user doesn't control that address (or it doesn't even exist).

The issue affects: ERC-4337 Account Abstraction wallets, Safe (Gnosis Safe) multisig wallets deployed on different chains, CREATE2-deployed wallets where deployment parameters differ across chains, and any smart contract wallet system that doesn't guarantee identical addresses across all chains. While EOA (Externally Owned Account) wallets have the same address across all chains (derived from the same private key), the growing adoption of smart contract wallets makes this a significant issue.

Impact

Users with Account Abstraction wallets or smart contract wallets cannot successfully use the cross-chain exchange if their wallet addresses differ across chains. When they deposit tokens on the source chain, the tokens are sent to their address on the



destination chain, but they cannot access those tokens because they don't control that address on the destination chain. This results in permanent loss of funds for the user. As Account Abstraction adoption grows (especially with ERC-4337 becoming more popular), this will affect an increasing percentage of users. The protocol would exclude a growing segment of the user base from using cross-chain features. Users might not discover this limitation until after they've already initiated a transaction and lost funds. This creates a poor user experience and potential reputational damage as users report lost funds. While the issue doesn't affect EOA users (currently the majority), planning for AA compatibility is important for future-proofing the protocol.

Recommendation

Add an optional destination receiver parameter to the `depositExchange` function to allow users to specify a different receiving address on the destination chain. Modify the function signature to include: `address destinationReceiver`. In the function logic, determine the recipient: `address recipient = destinationReceiver != address(0) ? destinationReceiver : msg.sender;`. Store the recipient in the transaction info instead of `msg.sender`: `txnInfos[txnId] = TxnInfo(txnId, recipient, tokenIn, ...);`. Update the signature verification to include the destination receiver in the signed message to prevent attackers from redirecting other users' funds. For backward compatibility with EOA users, if `destinationReceiver` is `address(0)`, default to using `msg.sender` (same behavior as current implementation). Add validation to prevent obvious misuse: `require(destinationReceiver != address(this), "Cannot send to contract");`. Update documentation and UI to explain the destination receiver parameter and when users should use it. This solution provides flexibility for AA wallet users while maintaining backward compatibility for EOA users.

Status

Acknowledged

[M-06] Exchange - Missing Validation for Signature Expiration Time

Description

The `setTimeExpiredSignature` function allows setting the signature expiration time to any value including zero, potentially causing denial of service or security issues.

Vulnerability Details

The `setTimeExpiredSignature` function at lines 145-147 allows operators to set the `timeExpiredSignature` variable to any `uint256` value without validation: `function setTimeExpiredSignature(uint256 time) external onlyOperator { timeExpiredSignature = time; }`. This variable controls how long signatures remain valid and is used in the timestamp validation at line 307. Setting it to inappropriate values creates problems.

If set to zero, the validation `block.timestamp - timestamp <= timeExpiredSignature` at line 307 becomes `block.timestamp - timestamp <= 0`, which means `timestamp >= block.timestamp`. Combined with the check at line 305 requiring `timestamp <= block.timestamp`, transactions would need `timestamp == block.timestamp` exactly, which is nearly impossible to achieve. Any transaction would fail, causing complete denial of service. If set to a very small value (e.g., 1 second), signatures expire almost immediately, making it nearly impossible for users to submit transactions successfully, especially during network congestion. If set to an extremely large value (e.g., years), signatures remain valid indefinitely, reducing security and enabling replay attacks over long time periods.

The initial value is set to 30 minutes (line 55), which seems reasonable, but there's nothing preventing an operator from changing it to problematic values. An operator mistake, a compromised operator, or a misunderstanding of the parameter's impact could cause the `timeExpiredSignature` to be set incorrectly, disrupting the entire exchange operation. While operators are trusted, enforcing reasonable bounds is a best practice to prevent accidental misconfiguration.

Impact

Setting `timeExpiredSignature` to zero or an extremely small value causes complete denial of service where no user can successfully execute cross-chain swaps. All transactions would fail with "Signature is expired" or "Timestamp is invalid" errors. Users would be unable to access the exchange functionality, wasting gas on failed transactions. The protocol would be effectively non-operational until an operator notices and corrects the setting. Setting it to an extremely large value (e.g., `type(uint256).max`) weakens security by allowing signatures to be valid indefinitely, increasing replay attack risks and reducing the freshness guarantees of signed messages. The lack of bounds checking creates fragility where a simple operator error can break the entire system. Even well-intentioned operators might make mistakes when dealing with time values, especially when converting between different time units (seconds, minutes, hours). The impact is denial of service (if set too small) or weakened security (if set too large).

Recommendation

Add reasonable bounds validation to the `setTimeExpiredSignature` function to prevent misconfiguration. Implement minimum and maximum limits based on operational requirements:

```
function setTimeExpiredSignature(uint256 time) external  
onlyOperator { require(time >= 5 minutes, "Time too short, must be at least 5  
minutes"); require(time <= 24 hours, "Time too long, must be at most 24 hours");  
timeExpiredSignature = time; }
```

The specific bounds should be chosen based on the protocol's requirements: the minimum should be long enough to account for network congestion and block times across different chains, while the maximum should be short enough to maintain security and prevent stale signatures. Consider typical use cases: users generating signatures and submitting within minutes, potential network delays and congestion, and security requirements for signature freshness. Add events when the time is updated: `emit TimeExpiredSignatureUpdated(oldValue, time);`. Document the chosen bounds in comments explaining the reasoning. Test the bounds thoroughly to ensure they work across different network conditions. This simple validation prevents configuration errors from breaking the system while maintaining operator flexibility within safe ranges.

Status

Acknowledged

[M-07] Proposal - Missing OnlyInitialized modifier

Description

`Proposal.createProposal` and `Proposal.voteProposal` are callable before system initialization. Without an `onlyInitialized` guard, proposals get a zero voting window and initial approvals (pass) are not seeded, breaking governance.

Vulnerability Details

`Proposal.initialize(address[] vals)` wires state and sets a 7-day voting window and initial approvals:

```
function initialize(address[] calldata vals) external onlyNotInitialized {  
  
    // set to a 7 days limit standard  
  
    proposalLastingPeriod = 7 days;  
  
    validators = Validators(ValidatorContractAddr);  
  
    for (uint256 i = 0; i < vals.length; i++) {  
  
        require(vals[i] != address(0), "Invalid validator address");  
  
        pass[vals[i]] = true;  
  
    }  
  
    initialized = true;  
  
}
```

However, both governance entry points lack an `onlyInitialized` gate.

Impact

Consequence before `initialize()` runs pass mapping for genesis validators remains unset, so proposals can be created for addresses that should already be authorized.

Recommendation

Add `onlyInitialized` to both `createProposal` and `voteProposal`.

Status

Acknowledged

[M-08] Validators.sol - Stakers array can be bloated with Owei stakes disallowing others to stake in a specific validator

Description

Zero/tiny stakes are accepted after the validator reaches the minimum 54 ETH, causing unbounded growth of the stakers array with zero-balance entries and duplicates. This is a storage/gas griefing vector that can also make getters expensive or revert when called on-chain by other contracts.

Vulnerability Details

In `Validators.stake`, once a validator's total stake `valInfo.coins` has met `MinimalStakingCoin` (54 ether), additional stakes are permitted with any `msg.value` (including zero):

```
`require(valInfo.coins.add(staking) >= MinimalStakingCoin, "Staking coins not enough");`
```

The contract then appends the staker to `valInfo.stakers` when it believes it is the staker's "first time," which is determined solely by `staked[staker][validator].coins == 0`:

```
if (staked[staker][validator].coins == 0) {  
    staked[staker][validator].index = valInfo.stakers.length;  
    valInfo.stakers.push(staker);  
}
```

Because there is no `require(msg.value > 0)`, an attacker can call `stake(validator)` repeatedly with `msg.value == 0`. Each call:

- Pushes the attacker address into stakers (since coins remains 0),
- Leaves `staked[attacker][validator].coins` at 0, and Creates immutable, duplicate, zero-balance entries that cannot be cleaned via `unstake` (which requires `coins > 0`).

- Current core logic does not iterate stakers, so no immediate DoS on protocol flows; however, the array is returned by `getValidatorInfo`, making reads increasingly expensive and risky for on-chain consumers.

Impact

- Storage bloat: Unbounded growth of stakers increases state size and long-term storage costs.
- Broken invariants: The single index per staker cannot represent multiple positions if duplicates exist; entries with zero balance are permanent (cannot unstake).

Recommendation

Disallow zero-value stakes and gate list insertion by positive stake

Status

Acknowledged

[M-09] Validators.sol - No duplicate checks on active validator set updates cause reward manipulation (both roots and v1)

Description

`Validators.updateActiveValidatorSet` accepts duplicate addresses and assigns them directly to `currentValidatorSet`, causing reward misallocation and incorrect validator counting for the epoch.

Vulnerability Details

Both `Validators.sol` (root) and `v1/Validators.sol` replace the active validator set without enforcing uniqueness or non-zero addresses. This allows duplicated entries to be counted multiple times in totals and rewards.

Root Version:

```
function updateActiveValidatorSet(address[] memory newSet, uint256 epoch)

    public

    onlyMiner

    onlyNotUpdated

    onlyInitialized

    onlyBlockEpoch(epoch)
{
    operationsDone[block.number][uint8(Operations.UpdateValidators)] = true;

    require(newSet.length > 0, "Validator set empty!");

    currentValidatorSet = newSet;

    emit LogUpdateValidator(newSet);
```

```
}

```

V1 Version:

```
function updateActiveValidatorSet(address[] memory newSet, uint256 epoch)

    external

    onlyMiner

    onlyNotOperated(Operation.UpdateValidators)

    onlyBlockEpoch(epoch)

    onlyInitialized

{
    operationsDone[block.number][Operation.UpdateValidators] = true;

    // assigns activeValidators = newSet; no deduplication
}
```

Downstream logic in both versions iterates the set by index, not by unique validator, so duplicates are counted multiple times in totals and rewards.

Total Length Accumulation

```
function getTotalStakeOfActiveValidatorsExcept(address val)

    private

    view

    returns (uint256 total, uint256 len)

{
    for (uint256 i = 0; i < currentValidatorSet.length; i++) {
        if (
            validatorInfo[currentValidatorSet[i]].status != Status.Jailed &&
            val != currentValidatorSet[i]
        ) {
```



```

        total = total.add(validatorInfo[currentValidatorSet[i]].coins);

        len++;
    }
}

```

As a result, a duplicated validator receives multiple reward slices per block and inflates totals, distorting proportional allocation for the entire epoch.

Impact

- Direct funds misallocation to duplicated validators for the epoch, reducing others' rewards.
- Incorrect validator counts (len) and any logic depending on index-based iteration.
- Scope is limited to epoch updates, but effects persist until the next set update.

Severity is Medium because a privileged but realistic actor (epoch miner) can trigger it in one call, and it directly impacts monetary distribution.

PoC

```

// called by miner at an epoch block

validators.updateActiveValidatorSet([A, A, B], epoch);

// next block reward distribution of 300

validators.distributeBlockReward{value: 300 ether}();

// Expected outcome:

// Stake sum counts duplicates: 100 (A) + 100 (A again) + 100 (B) = 300

// Allocations per entry:

// A gets 300 * 100 / 300 twice = 200 total

```

```
// B gets 300 * 100 / 300 once = 100 total  
  
// A wrongfully earns 66.7% instead of 50%.
```

Recommendation

Enforce uniqueness and non-zero addresses inside `updateActiveValidatorSet` in both files. Optionally require each address to be authorized and not jailed.

Status

Acknowledged

[M-10] Validators.sol - Miner can bypass epoch gating and neutralize punishment within the same block

Description

The epoch restriction in `decreaseMissedBlocksCounter` relies on a caller-supplied `epoch` argument checked as `block.number % epoch == 0`. A miner can pass `epoch = 1` to make the function callable every block. Additionally, `punish` and `decreaseMissedBlocksCounter` use separate once-per-block guards, allowing both operations in the same block. This enables a miner to increment a validator's missed counter and immediately decrease it in the same block, preventing thresholds from being reached.

Vulnerability Details

In `Validators.stake`, once a validator's total stake `valInfo.coins` has met `MinimalStakingCoin` (54 ether), additional stakes are permitted with any `msg.value` (including zero):

```
`require(valInfo.coins.add(staking) >= MinimalStakingCoin, "Staking coins not enough");`
```

The contract then appends the staker to `valInfo.stakers` when it believes it is the staker's "first time," which is determined solely by `staked[staker][validator].coins == 0`:

```
if (staked[staker][validator].coins == 0) {  
    staked[staker][validator].index = valInfo.stakers.length;  
    valInfo.stakers.push(staker);  
}
```

Because there is no `require(msg.value > 0)`, an attacker can call `stake(validator)` repeatedly with `msg.value == 0`. Each call:

- Pushes the attacker address into `stakers` (since `coins` remains 0),

- Leaves `staked[attacker][validator].coins` at `0`, and Creates immutable, duplicate, zero-balance entries that cannot be cleaned via `unstake` (which requires `coins > 0`).
- Current core logic does not iterate stakers, so no immediate DoS on protocol flows; however, the array is returned by `getValidatorInfo`, making reads increasingly expensive and risky for on-chain consumers.

Impact

- Storage bloat: Unbounded growth of stakers increases state size and long-term storage costs.
- Broken invariants: The single index per staker cannot represent multiple positions if duplicates exist; entries with zero balance are permanent (cannot unstake).

Recommendation

Disallow zero-value stakes and gate list insertion by positive stake

Status

Acknowledged

[M-11] v1/library/SortedList - Missing OnlyInitialized modifier

Description

The SortedList library uses an old solidity compiler version that lacks automated overflow checks. The sorted linked list used for validator ranking stores length as uint8. Insertion and removal adjust this counter with unchecked arithmetic. Once the list reaches 255 elements, length++ wraps to 0; subsequent operations may treat the list as empty and reset 'head'/'tail', corrupting the ranking structure.

Vulnerability Details

Length is 'uint8'; inserts increment and removals decrement without bounds:

```
// v1/library/SortedList.sol

struct List {

    IVotePool head;

    IVotePool tail;

    uint8 length;

    mapping(IVotePool => IVotePool) prev;

    mapping(IVotePool => IVotePool) next;

}

function improveRanking(List storage _list, IVotePool _value) internal {

    // insert into empty list

    if (_list.length == 0) {

        _list.head = _value;

        _list.tail = _value;

        _list.length++; // wraps from 255 -> 0 elsewhere

        return;

    }

}
```

```

IVotePool _prev = _list.prev[_value];

if (_prev == IVotePool(0)) {

    _list.length++; // unchecked increment, uint8 overflow at 255

    // ... link insertion logic ...

}

// ...

}

function removeRanking(List storage _list, IVotePool _value) internal {

    // ... unlink logic ...

    _list.length--; // unchecked decrement; underflows if already 0

}

```

- The list is used in `v1/Validators.sol` to maintain top vote pools per validator type (topVotePools). There is no hard cap on the number of pools; `initialize` and `addValidator` can create many `VotePool` instances, and pools call `improveRanking()`.
- Overflow scenario: At 255 elements, inserting a new pool does `length++` and wraps to 0 mid-function. On a later call, the leading `if (_list.length == 0)` path fires and resets `'head'/'tail'` to the new value, orphaning the existing chain and corrupting ordering.

Impact

- Corrupted ranking list can break selection of backups/top validators and distort any logic relying on ordering, causing incorrect validator composition and reward allocations derived from rankings.
- Once corrupted, the structure may be difficult to recover without manual intervention or full rebuild. Even without malicious intent, growth can trigger overflow.

Recommendation

- Make ``length`` a wider type (e.g., ``uint16`` or ``uint256``) to remove practical overflow risk.
- Use the latest Solidity Compiler version

Status

Acknowledged

[M-12] `adil-service-contracts/AdilServicePharmacy` - Incorrect Asset Accounting with Fee-on-Transfer Tokens Leads to Permanent Loss of Funds

Description

The `makeOrder` function's accounting logic incorrectly assumes that the `_amount` specified in the function parameters is the exact amount of tokens that the contract will receive. This assumption is violated by fee-on-transfer (FOT) tokens, which deduct a fee during the `transferFrom` operation. The contract saves the user-specified `_amount` to the `Order` struct without verifying the actual amount received. This creates a discrepancy between the contract's internal state and its real token balance.

Vulnerability Details

This flaw leads to a direct and permanent loss of user funds. If a user creates an order with a FOT token and later attempts to cancel it by calling `cancelOrder`, the transaction will always revert. The `cancelOrder` function tries to refund the originally stored `order.amount`:

```
function cancelOrder(uint256 _orderId, uint256 _timestamp, bytes calldata signature)
external signatureIsExpired(_timestamp) {

    //... rest of the code

    require(IERC20(order.token).balanceOf(address(this)) >= order.amount,
"ADILServicePharmacy: Insufficient contract balance");

    order.status = OrderStatus.Cancelled;

    TransferHelper.safeTransfer(order.token, msg.sender, order.amount); //<@ This
amount is not the actual amount that was received in case of an FOT token

}
```

but the contract holds a smaller balance due to the transfer fee. Because the refund amount is greater than the contract's balance for that order, the transfer fails, and the user's tokens become permanently and irrevocably trapped in the contract.

Impact

This will lead to a direct loss of funds to the users the minute Protocol allows using signatures for any FOT Token

Recommendation

The contract must not trust the `_amount` parameter. Instead, it should adopt a "balance-diffing" approach to determine the actual amount of tokens received. The contract should record its token balance before the transfer and after the transfer, and then store the difference as the true order amount.

Status

Acknowledged

[M-13] Tokenize - Precision Loss and Mathematical Rounding Errors in Price Calculations

Description

Multiple mathematical operations throughout the contract use integer division which causes rounding down, potentially leading to cumulative value loss in swap calculations.

Vulnerability Details

The contract performs several division operations that are subject to Solidity's integer division behavior, which always rounds down toward zero. At line 224, the `newPrice` calculation uses `PRICE_PRECISION.mul(PRICE_PRECISION).div(price)`, which is `1e18 / price`. For certain price values that don't divide evenly into `1e18`, the result will be rounded down, losing precision. More significantly, at line 287 in the `getRateToken` function, the token-to-token rate calculation uses `tokenPrices[from].mul(PRICE_PRECISION).div(tokenPrices[to])`. When `tokenPrices[to]` is large, this division can lose significant precision. In the `swap` function at line 325-327, the `amountOut` calculation uses `(amountInSubFee.mul(rateToken)).div(PRICE_PRECISION)`, which rounds down, meaning users receive slightly less than the mathematically correct amount. The fee calculation at line 318 uses `amountIn.mul(swapInfo.fee).div(PRICE_PRECISION)`, which also rounds down, but in this case it benefits the user slightly and disadvantages the protocol. The cumulative effect of multiple rounding operations in `calcAmountIn` function (lines 423-425) can compound the precision loss. The root cause is the inherent limitation of integer arithmetic in Solidity combined with the use of `PRICE_PRECISION` (`1e9`) which may not provide enough decimal places for certain calculations, especially when dealing with tokens that have 18 decimals. While each individual rounding may only lose a few wei, high-frequency trading or large volume swaps could amplify these losses significantly.

Impact

The precision loss from integer division creates several problematic scenarios that affect users and the protocol differently. Users consistently receive slightly less output tokens

than they should mathematically receive due to rounding down in the amountOut calculation. For small trades, this loss might be negligible (a few wei), but for large trades or high-frequency trading, the cumulative loss can become substantial. Consider a user making 1000 swaps per day - even losing 1 wei per transaction results in 1000 wei lost daily, and with larger amounts the losses scale proportionally. The protocol also loses fee revenue due to rounding in the fee calculation, though this is typically smaller than user losses. The bidirectional calculation functions (`calcAmountOut` and `calcAmountIn`) may produce inconsistent results due to different rounding behaviors - calculating the input needed for a desired output and then using that input to calculate output may not yield the original desired amount due to compounding rounding errors. This asymmetry can be exploited by sophisticated users who understand the rounding patterns to minimize their costs or maximize their outputs. Arbitrageurs could potentially exploit these rounding differences across multiple swaps to extract value from the protocol. The issue is particularly severe when swapping between tokens with very different decimal precisions or very different price scales, where the rounding errors become more pronounced and noticeable.

Recommendation

Address precision loss issues through a multi-faceted approach. First, consider increasing `PRICE_PRECISION` from `1e9` to `1e18` to provide more decimal places for price calculations, though this requires careful testing to ensure no arithmetic overflows occur in multiplication operations. Second, implement rounding-up logic for calculations that favor users (like `amountOut`) to ensure users never receive less than entitled, changing line 325-327 to:

```
uint256 amountOut =
(amountInSubFee.mul(rateToken).add(PRICE_PRECISION.sub(1))).div(PRICE_PRECISION)
```

; This adds `(PRICE_PRECISION - 1)` before division, effectively rounding up instead of down. Third, implement a dust threshold mechanism where very small rounding differences (below a certain wei threshold) are tolerated but larger discrepancies trigger warnings or reverts. Fourth, add comprehensive unit tests that specifically test edge cases with prices and amounts that are known to cause maximum rounding errors, ensuring the losses are within acceptable bounds. Consider implementing a small buffer or tolerance in the slippage check to account for rounding differences. Document all rounding behaviors clearly in the code comments and user documentation so users

understand they may receive slightly less than calculated due to rounding. For critical calculations, consider using a well-audited fixed-point arithmetic library like ABDKMath64x64 or PRBMath that provides higher precision decimal operations. Finally, implement monitoring to track cumulative rounding losses over time to ensure they remain within acceptable limits and don't indicate a larger mathematical error.

Status

Acknowledged

[M-14] ADILGameServiceAdilVault#withdraw - Unrestricted Fund Withdrawal in ADILGameServiceAdilVault

Description

Users can withdraw arbitrary amounts from the vault without any verification of their deposit amount.

Vulnerability Details

The `withdraw` function in `ADILGameServiceAdilVault` allows users to withdraw any amount from the contract as long as they have a valid signature from the off-chain signer. The vulnerability lies in the fact that the contract does not track individual user deposits or verify that the withdrawal amount corresponds to what the user has actually deposited. The function only checks if the withdrawal ID has been used before and if the signature is valid, but it does not validate whether the user is entitled to withdraw the requested amount.

The vulnerable code at lines 49-51 shows that the function accepts `_amount` as a parameter and only verifies it against the total contract balance, not against the user's individual deposit balance. An attacker who obtains a single valid signature (either through legitimate means or by compromising the off-chain signing service) can drain the entire vault by calling the `withdraw` function with the maximum available balance. Since the contract uses a string-based ID system (`isWithdrawn[_id]`) rather than tracking per-user balances, there is no mechanism to prevent a user from withdrawing more than they deposited. The signature validation only confirms that the signer approved the transaction parameters, but it doesn't enforce any business logic about withdrawal limits based on deposits.

Impact

This is a critical vulnerability that allows any user with a valid signature to steal the entire balance of the vault. Since deposits from multiple users are pooled in the contract without individual accounting, a malicious user could withdraw funds deposited by others. This completely breaks the trust model of the vault system and could result in total loss of all deposited funds. The economic impact is severe as it affects all users

who have deposited native tokens into the vault, and the attacker only needs to obtain or generate one valid signature to exploit this vulnerability

Recommendation

Implement a comprehensive deposit tracking system that maintains individual user balances. The contract should use a mapping structure such as `mapping(address => uint256) public userBalances` to track how much each user has deposited. When a user deposits funds, their balance should be incremented accordingly. During withdrawal, the contract must verify that the requested withdrawal amount does not exceed the user's available balance. Additionally, consider implementing a two-step withdrawal process where users first request a withdrawal that is then approved by the off-chain system, with the signature encoding both the user address and their maximum allowed withdrawal amount based on their deposit history. The withdraw function should include a check like `require(userBalances[msg.sender] >= _amount, "Insufficient user balance")` before processing the transfer.

Status

Acknowledged

[M-15] Exchange#depositExchange - Timestamp Logic Error Causes Transaction Failures

Description

The timestamp validation logic is inverted, causing legitimate transactions to fail and creating denial of service conditions.

Vulnerability Details

The `depositExchange` function contains fundamentally flawed timestamp validation logic at lines 305-308. The first check at line 305 requires `timestamp <= block.timestamp`, meaning the timestamp parameter must be less than or equal to the current block timestamp. This logic is backwards from typical signature expiration patterns. The second check at line 307 requires `block.timestamp - timestamp <= timeExpiredSignature`, attempting to ensure the signature isn't too old.

The problems with this logic are: First, the requirement `timestamp <= block.timestamp` means timestamps must be in the past or present, never in the future. In typical signature schemes, timestamps represent when the signature expires (a future time), but this implementation treats them as when the signature was created (a past time). Second, the combination of both checks creates a very narrow validity window. For a transaction to succeed, its timestamp must be exactly at or slightly before `block.timestamp`, but not too far in the past (within `timeExpiredSignature`). Third, this creates race conditions where a transaction that was valid when submitted becomes invalid by the time it's mined if even one block passes.

The practical impact: When a user requests a signature from the off-chain service, the service generates a signature with a timestamp. If that timestamp is set to the current time or slightly in the future (standard practice for expiration times), the transaction will immediately fail with "Timestamp is invalid" when submitted on-chain. If the timestamp is set to the current time and the transaction takes several seconds or a block to be mined, it might fail the expiration check. Users experience constant transaction failures despite having valid signatures. The only way transactions succeed is if the timestamp is set to past time and the transaction is mined within the expiration window, which is fragile and error-prone.

The comment in the findings suggests the correct logic should be: `require(timestamp >= block.timestamp, "Timestamp too old")` to ensure the signature hasn't expired yet, and `require(timestamp - block.timestamp <= timeExpiredSignature, "Signature expired")` to ensure the expiration time isn't too far in the future. This would create a standard signature expiration pattern where timestamps represent future expiration times.

Impact

This vulnerability causes a denial of service condition where legitimate users cannot successfully execute cross-chain swaps despite having properly signed transactions. The broken timestamp logic creates constant transaction failures, making the exchange unusable or extremely unreliable. Users waste gas on failed transactions, experience frustration, and lose confidence in the protocol. The narrow validity window makes successful transactions dependent on perfect timing and low network congestion, which is impractical for a production system. During high network congestion when transactions take longer to mine, the failure rate increases dramatically. This essentially breaks the core functionality of the exchange, making it unable to fulfill its intended purpose. While not a direct fund loss vulnerability, the inability to process transactions makes the protocol non-functional and could lead to indirect losses if users' time-sensitive arbitrage opportunities are missed or if they're forced to use alternative more expensive services.

Recommendation

Reverse the timestamp validation logic to implement standard signature expiration behavior. Replace lines 305-308 with corrected logic: `require(timestamp >= block.timestamp, "Signature already expired"); require(timestamp - block.timestamp <= timeExpiredSignature, "Expiration time too far in future");`. This implements the intended behavior where `timestamp` represents when the signature expires (a future time), the first check ensures the signature hasn't already expired, and the second check ensures users can't create signatures that are valid for unreasonably long periods. Update the off-chain signing service to generate timestamps as `currentTime + validityPeriod` rather than just `currentTime`. For example, if the signature should be valid for 30 minutes, set `timestamp = block.timestamp + 30`

minutes. Consider adding additional comments in the code to clarify the timestamp semantics: "timestamp represents the expiration time of the signature, must be in the future but not more than timeExpiredSignature seconds ahead". Test thoroughly with various timestamp values to ensure transactions succeed within the intended validity window and fail appropriately outside it.

Status

Acknowledged

[M-16] Tokenize#swap - Flawed Access Control Logic in Swap Function

Description

The swap function contains a logical flaw in its access control mechanism that prevents all users (including admins) from executing swaps when both `isAdminSwap` and `isUserSwap` flags are set to true.

Vulnerability Details

The vulnerability exists in the swap function at lines 305-309 in `Tokenize.sol:305`. The access control logic uses a mutually exclusive if-else structure: `if (swapInfo.isAdminSwap) { require(admins[msg.sender], "Owner can't swap token"); } else { require(swapInfo.isUserSwap, "User can't swap token"); }`. This logic assumes that `isAdminSwap` and `isUserSwap` cannot both be true simultaneously. However, the `SwapInfo` struct allows both boolean flags to be set to true when an operator calls `setSwapInfo`. When both flags are true, the function enters the first branch (`isAdminSwap` condition), which requires the caller to be an admin. This means regular users cannot swap even though `isUserSwap` is true. Conversely, if the intention was for both admins and users to be able to swap when both flags are true, the current logic still only allows admins. The root cause is the use of an if-else structure instead of independent checks for each flag, combined with unclear semantics about what it means when both flags are true. The error messages are also confusing - "Owner can't swap token" when it should say "Only admins can swap" and "User can't swap token" when it should say "Swapping is not enabled for users".

Impact

This vulnerability creates significant operational issues and potential loss of functionality for the protocol. Depending on how operators configure the swap pairs, certain swap routes could become completely inaccessible or accessible only to a restricted group. If an operator mistakenly sets both flags to true expecting to enable swaps for everyone, regular users will be blocked from swapping, leading to user complaints and potential loss of trading volume. The protocol's flexibility is severely limited as there's no way to configure a swap pair that allows both admins and regular users to swap with different conditions or permissions. This could be problematic for

emergency situations where admins need to maintain access while gradually opening or closing access to users. The confusing error messages will make debugging difficult, as users and developers may not understand why swaps are failing when the `isUserSwap` flag appears to be enabled. Smart contract integrations that depend on predictable access control behavior may malfunction when encountering swap pairs configured with both flags set to true. This could lead to failed transactions, locked funds in intermediary contracts, or incorrect state assumptions in dependent protocols.

Recommendation

The access control logic should be redesigned to handle all possible flag combinations clearly and explicitly. There are several approaches to fix this issue. First approach: Make the flags mutually exclusive by adding a check in the `setSwapInfo` function: `require(!(swapInfo.isAdminSwap && swapInfo.isUserSwap), "Cannot enable both admin and user swap simultaneously");` This ensures only one flag can be true at a time, maintaining the current if-else logic but preventing the problematic state. Second approach: Redesign the logic to support both flags being true, allowing both admins and users to swap: `if (swapInfo.isAdminSwap && swapInfo.isUserSwap) { // Allow both admins and users } else if (swapInfo.isAdminSwap) { require(admins[msg.sender], "Only admins can swap this pair"); } else if (swapInfo.isUserSwap) { // Allow all users } else { revert("Swapping is disabled for this pair"); }`. Third approach: Use independent checks: `require(swapInfo.isAdminSwap || swapInfo.isUserSwap, "Swapping is disabled"); if (!swapInfo.isUserSwap) { require(admins[msg.sender], "Only admins can swap this pair"); }`. Additionally, improve error messages to be more descriptive and accurate about the actual access control requirements. Document the intended behavior of the flags clearly in the code comments and protocol documentation.

Status

Acknowledged

[M-17] Tokenize#setADILPrice - Incorrect Token-to-Token Swap Rate Calculation Due to Uniform Pricing

Description

The `setADILPrice` function sets the same price for all three tokenized assets (MARE, EMAS, and ROY), making token-to-token swaps meaningless and returning incorrect exchange rates.

Vulnerability Details

In the `setADILPrice` function at lines 227-229, all three token addresses (`mareAddress`, `emasAddress`, and `royAddress`) are assigned the exact same `newPrice` value using `setTokenPrice`. The `newPrice` is calculated as `PRICE_PRECISION.mul(PRICE_PRECISION).div(price)` which represents the inverse of the ADIL price. This means that regardless of the actual market value or intended value of MARE, EMAS, or ROY tokens, they all receive identical pricing. The consequence of this becomes apparent in the `getRateToken` function at line 287, where token-to-token swap rates are calculated using the formula: `tokenPrices[from].mul(PRICE_PRECISION).div(tokenPrices[to])`. Since `tokenPrices[from]` and `tokenPrices[to]` are always equal for any token pair, this calculation always returns exactly `PRICE_PRECISION` (1e9) regardless of which tokens are being swapped. The root cause is the architectural decision to derive all token prices solely from the ADIL price without any mechanism to differentiate between different tokenized assets. This suggests that MARE, EMAS, and ROY are intended to be equivalent to each other, but if that's the case, having three separate tokens is unnecessary and confusing.

Impact

This design flaw fundamentally breaks the token swap mechanism for any token-to-token exchanges within the protocol. Users who attempt to swap between MARE, EMAS, and ROY tokens will always receive a 1:1 exchange rate regardless of the actual market values or intended values of these tokens. If these tokens are meant to represent different assets (such as different real-world commodities or securities), their prices should vary based on their underlying asset values, but the current

implementation makes this impossible. This could lead to significant economic exploits where users could arbitrage between tokens that should have different values but are forced to trade at parity. For example, if MARE should be worth twice as much as EMAS in real-world value, users could swap EMAS for MARE at 1:1, effectively doubling their value. The protocol has no way to adjust individual token prices independently, making it impossible to accurately represent the real-world values of different tokenized assets. This limitation severely restricts the protocol's usefulness and could lead to substantial financial losses for users who expect fair market pricing.

Recommendation

The protocol should implement independent price oracles or pricing mechanisms for each tokenized asset (MARE, EMAS, and ROY) rather than deriving all prices from a single ADIL price. This can be achieved by removing lines 227-229 from the `setADILPrice` function and instead creating separate price-setting functions for each token, such as `setMAREPrice`, `setEMASPrice`, and `setROYPrice`, each callable by operators with appropriate access controls. Alternatively, implement a price oracle integration that fetches real-time prices for each asset from trusted external sources. The `setADILPrice` function should only set the ADIL price and not automatically update other token prices. If the three tokens are intended to be equivalent and pegged to each other, then the protocol should clearly document this design decision and consider whether having three separate tokens is necessary, or if a single token would be more appropriate. If the tokens must remain pegged, implement explicit peg mechanisms with clear documentation explaining why all tokens have the same value and under what conditions this peg might change.

Status

Resolved

Low

[L-01] OwnerOperator - Use of Ownable Instead of Ownable2Step

Description

The contract inherits from OpenZeppelin's Ownable instead of the safer Ownable2Step pattern.

Vulnerability Details

The OwnerOperator contract at line 6 inherits from @openzeppelin/contracts/access/Ownable.sol, which implements a single-step ownership transfer mechanism. When the owner calls `transferOwnership(newOwner)`, ownership is immediately transferred to the new address without any confirmation from the recipient. This creates a risk if the owner accidentally passes an incorrect address (typo, wrong address copied, etc.) or if the new owner address is a contract that cannot handle ownership (no owner management functions). Once ownership is transferred to such an address, it cannot be recovered, and all owner-only and operatorOrOwner functions become permanently inaccessible.

The Ownable2Step pattern (available in OpenZeppelin Contracts v4.9.0+) implements a two-step process where the current owner first nominates a new owner, and then the nominated address must accept ownership by calling a separate function. This prevents accidental transfers to wrong addresses because the recipient must actively claim ownership, confirming they can interact with the contract.

Impact

Accidental transfer of ownership to an incorrect or inaccessible address would result in permanent loss of administrative control over the contracts. This would prevent critical operations such as adding/removing operators, updating signer addresses, pausing contracts, managing distributions, and withdrawing funds.

The protocol could be left in a permanently locked state where no administrative actions can be taken.

Recommendation

Replace the Ownable import with Ownable2Step from OpenZeppelin Contracts. Change line 4 from `import "@openzeppelin/contracts/access/Ownable.sol";` to `import "@openzeppelin/contracts/access/Ownable2Step.sol";` and update line 6 to inherit from Ownable2Step instead: `abstract contract OwnerOperator is Ownable2Step`. Ensure that the OpenZeppelin Contracts dependency is updated to at least version 4.9.0 to have access to Ownable2Step. After making this change, test the ownership transfer process to ensure that both steps (propose and accept) work correctly. Update documentation and operational procedures to reflect the two-step process for ownership transfers. This change significantly reduces the risk of accidental ownership loss with minimal code changes.

Status

Acknowledged

[L-02] Exchange - Unchecked ERC20 Transfer Return Value

Description

The `transferFrom` call for ERC20 tokens does not check the return value, potentially allowing failed transfers to go undetected.

Vulnerability Details

At line 335 in the `depositExchange` function, the code executes `erc20Token.transferFrom(msg.sender, adminReceiver, amountIn)` without checking the return value. While many modern ERC20 tokens revert on failure, the ERC20 standard technically allows implementations that return `false` to indicate failure rather than reverting. Some tokens, particularly older implementations or non-standard tokens, follow this pattern. The code casts `tokenIn` to `IERC20` interface and calls `transferFrom`, but the boolean return value is ignored.

If a token returns `false` instead of reverting, the code continues execution as if the transfer succeeded. The cross-chain swap record gets created (lines 362-372), the transaction info is stored, and the `DepositExchange` event is emitted. From the system's perspective, the user has successfully deposited tokens for a cross-chain swap. However, no tokens were actually transferred from the user to the `adminReceiver`. On the destination chain, when `withdrawExchange` is called, the user receives the full `amountOut` despite never actually depositing the required `amountIn`.

Impact

If a non-standard ERC20 token that returns `false` on transfer failure is used, attackers could exploit this to receive tokens on the destination chain without actually depositing tokens on the source chain. This creates a loss for the protocol equal to the value of tokens paid out on the destination chain.

Recommendation

Use OpenZeppelin's `SafeERC20` library which properly handles both reverting tokens and tokens that return `false`.

This is an industry standard best practice and should be applied consistently across all ERC20 interactions in the codebase.

Status

Acknowledged



[L-03] Exchange - Operator Privilege Abuse Risk in withdrawExchange

Description

The withdrawExchange function allows operators to provide arbitrary values without validation against stored deposit data, creating centralization and manipulation risks.

Vulnerability Details

The withdrawExchange function at lines 381-443 is callable only by operators (via the `onlyOperator` modifier at line 390) and accepts numerous parameters that define the withdrawal: sender address, token addresses, amounts, exchange info, and timestamp. The function creates a completely new TxnInfo struct from these operator-provided parameters at lines 391-401, rather than reading data from a corresponding deposit transaction.

The only validation performed is: checking that the txnid doesn't already exist (lines 402-405), verifying that the computed txnid matches the provided one (lines 406-414), and checking token balances/allowances (lines 428-436). Notably absent is any validation that these values correspond to an actual deposit transaction on the source chain, that the exchange rate is correct, or that the amounts are reasonable. A malicious or compromised operator could call withdrawExchange with fabricated values, creating fraudulent withdrawal records and draining contract funds.

Specific attack vectors include: An operator could specify inflated amountOut values to drain more tokens than appropriate. They could use incorrect exchange rates in the exchangeInfo parameter to give themselves or collaborators favorable swaps. They could create withdrawal transactions for addresses that never made deposits. They could process the same conceptual deposit multiple times with different txnids. The function assumes operators are completely trustworthy and will accurately relay information from the source chain, but provides no cryptographic proof or cross-validation mechanism.

Impact

A compromised operator or a malicious insider with operator privileges could drain all funds from the destination chain contracts by calling `withdrawExchange` with arbitrary amounts. Even an honest operator making a mistake (typo in amounts, wrong exchange rate, etc.) could cause loss of funds or unfair distributions. The centralized trust in operators represents a single point of failure for the entire cross-chain bridge. Users must trust that operators will accurately relay transaction information without manipulation. This centralization risk is contrary to the trustless ideals of blockchain systems. If operator private keys are compromised through phishing, social engineering, or security breaches, the attacker gains complete control over fund distributions. The lack of any cryptographic verification of cross-chain data makes the system vulnerable to operator-based attacks. While some level of operator trust is common in bridge designs, the complete absence of validation or limits makes this implementation particularly risky.

Recommendation

Implement validation mechanisms to limit operator discretion and verify the integrity of withdrawal data. Several approaches can improve security: First, use signature verification where users must sign approval for withdrawals on the destination chain, similar to how `depositExchange` uses signatures. Second, implement withdrawal limits where operators can only process withdrawals up to certain amounts per time period, requiring multi-sig or owner approval for large withdrawals. Third, add exchange rate validation by storing valid exchange rate ranges for token pairs and requiring that operator-provided rates fall within acceptable bounds. Fourth, implement a timelock mechanism where withdrawals are first queued with a delay, giving the owner time to review and cancel suspicious transactions. Fifth, emit detailed events and implement off-chain monitoring to detect anomalous operator behavior. Sixth, consider more advanced solutions like implementing a light client proof system or using oracle networks to verify cross-chain data. At minimum, add reasonableness checks such as maximum withdrawal amounts, required exchange rate ranges, and validation that the sender address is reasonable. Move toward a multi-operator system where multiple operators must agree on withdrawal parameters before execution, reducing single-point-of-failure risk.

Status

Acknowledged

[L-04] Tokenize - Missing Event Parameters and Incomplete Event Logging

Description

Several important state-changing functions do not emit events or emit incomplete events, making it difficult to track contract state changes off-chain.

Vulnerability Details

Multiple setter functions modify critical state variables but either do not emit events or emit events with insufficient information. The `setMAREAddress`, `setEMASAddress`, and `setROYAddress` functions (lines 245-263) modify token addresses but emit no events at all. This makes it impossible for off-chain systems to detect when these critical addresses are changed. The `setTokenPrice` function (line 210-215) also modifies pricing data without emitting an event, though it's called internally by `setADILPrice` which does emit a `ChangeTokenPrice` event. However, the `ChangeTokenPrice` event only includes a boolean parameter (`isChange`) which provides no useful information - it doesn't indicate which price changed, what the old value was, or what the new value is. The `setAdminFeeAddress` function (line 237-239) changes the fee recipient address without any event emission, making it impossible to audit fee address changes. The `removeSwapInfo` function (line 200-203) deletes swap configuration without logging what was removed. The `setSwapInfo` function (line 90-97) modifies swap configuration and calls `addSwapToken` internally, but doesn't emit an event to log this important change. Events are crucial for off-chain monitoring, analytics, and security systems. Without proper events, detecting unauthorized changes, debugging issues, or maintaining a full audit trail of contract state becomes extremely difficult.

Impact

The lack of comprehensive event emissions severely impacts the observability and auditability of the contract. Off-chain systems such as frontends, analytics platforms, and monitoring tools rely on events to track contract state changes in real-time.

Without events for token address changes, these systems cannot detect when token contracts are updated, which could lead to users interacting with outdated or incorrect token addresses. Security monitoring systems cannot detect unauthorized changes to critical parameters if those changes are not logged through events. For example, if an operator account is compromised and the attacker changes the `adminFeeAddress` to their own address, there's no event to trigger alerts in monitoring systems, allowing the attack to continue undetected until someone manually checks the contract state. The `ChangeTokenPrice` event's boolean parameter is essentially useless for analytics - off-chain systems cannot determine the actual price changes, making it impossible to build price history graphs or detect suspicious price manipulations. Auditors investigating historical contract behavior will have difficulty reconstructing the sequence of state changes because many changes are not logged. Users who want to verify that their transactions were processed correctly may not be able to find the necessary information in event logs. The absence of events also makes smart contract indexing and querying more difficult, as indexers typically rely on events rather than reading state directly. This impacts the user experience of any dapp built on top of this contract, as the dapp cannot efficiently update its UI in response to contract changes.

Recommendation

Implement comprehensive event emissions for all state-changing functions following Ethereum best practices. For token address setters, add events like: `event MAREAddressChanged(address indexed oldAddress, address indexed newAddress, uint256 timestamp);` `event EMASAddressChanged(address indexed oldAddress, address indexed newAddress, uint256 timestamp);` `event ROYAddressChanged(address indexed oldAddress, address indexed newAddress, uint256 timestamp);` and emit these events in the respective setter functions, capturing both old and new values. For the `setAdminFeeAddress` function, add: `event AdminFeeAddressChanged(address indexed oldAddress, address indexed newAddress, uint256 timestamp);`. Replace the current `ChangeTokenPrice` event with a more detailed version: `event TokenPriceChanged(address indexed token, uint256 oldPrice, uint256 newPrice, uint256 timestamp);` and emit this for each token price change individually rather than emitting a single boolean event. For `setSwapInfo` and `removeSwapInfo`, add events like: `event SwapInfoUpdated(address indexed from, address indexed to, uint256 fee,`

```
TypeFee    typeFee,    bool    isUserSwap,    bool    isAdminSwap);    event  
SwapInfoRemoved(address indexed from, address indexed to);
```

Make sure to emit events AFTER the state changes have been made to ensure the emitted values match the actual new state. Include indexed parameters for addresses to allow efficient filtering of events by token or address. Consider including a timestamp parameter in all events to make temporal analysis easier, though `block.timestamp` is always available in logs. Review all existing events to ensure they provide sufficient information for off-chain systems to fully reconstruct state changes without needing to query contract state directly.

Status

Acknowledged

Centralisation

The ADIL Chain project values security and utility over decentralisation.

The owner executable functions within the protocol increase security and functionality but depend highly on internal team responsibility.



Centralised

Decentralised

Conclusion

After Hashlock's analysis, the Adil Chain project seems to have a sound and well-tested code base, now that our vulnerability findings have been resolved or acknowledged. Overall, most of the code is correctly ordered and follows industry best practices. The code is well commented on as well. To the best of our ability, Hashlock is not able to identify any further vulnerabilities.

Our Methodology

Hashlock strives to maintain a transparent working process and to make our audits a collaborative effort. The objective of our security audits is to improve the quality of systems and upcoming projects we review and to aim for sufficient remediation to help protect users and project leaders. Below is the methodology we use in our security audit process.

Manual Code Review:

In manually analysing all of the code, we seek to find any potential issues with code logic, error handling, protocol and header parsing, cryptographic errors, and random number generators. We also watch for areas where more defensive programming could reduce the risk of future mistakes and speed up future audits. Although our primary focus is on the in-scope code, we examine dependency code and behaviour when it is relevant to a particular line of investigation.

Vulnerability Analysis:

Our methodologies include manual code analysis, user interface interaction, and white box penetration testing. We consider the project's website, specifications, and whitepaper (if available) to attain a high-level understanding of what functionality the smart contract under review contains. We then communicate with the developers and founders to gain insight into their vision for the project. We install and deploy the relevant software, exploring the user interactions and roles. While we do this, we brainstorm threat models and attack surfaces. We read design documentation, review other audit results, search for similar projects, examine source code dependencies, skim open issue tickets, and generally investigate details other than the implementation.

Documenting Results:

We undergo a robust, transparent process for analysing potential security vulnerabilities and seeing them through to successful remediation. When a potential issue is discovered, we immediately create an issue entry for it in this document, even though we have not yet verified the feasibility and impact of the issue. This process is vast because we document our suspicions early even if they are later shown to not represent exploitable vulnerabilities. We generally follow a process of first documenting the suspicion with unresolved questions, and then confirming the issue through code analysis, live experimentation, or automated tests. Code analysis is the most tentative, and we strive to provide test code, log captures, or screenshots demonstrating our confirmation. After this, we analyse the feasibility of an attack in a live system.

Suggested Solutions:

We search for immediate mitigations that live deployments can take and finally, we suggest the requirements for remediation engineering for future releases. The mitigation and remediation recommendations should be scrutinised by the developers and deployment engineers, and successful mitigation and remediation is an ongoing collaborative process after we deliver our report, and before the contract details are made public.

Disclaimers

Hashlock's Disclaimer

Hashlock's team has analysed these smart contracts in accordance with the best industry practices at the date of this report, in relation to: cybersecurity vulnerabilities and issues in the smart contract source code, the details of which are disclosed in this report, (Source Code); the Source Code compilation, deployment, and functionality (performing the intended functions).

Due to the fact that the total number of test cases is unlimited, the audit makes no statements or warranties on the security of the code. It also cannot be considered as a sufficient assessment regarding the utility and safety of the code, bug-free status, or any other statements of the contract. While we have done our best in conducting the analysis and producing this report, it is important to note that you should not rely on this report only. We also suggest conducting a bug bounty program to confirm the high level of security of this smart contract.

Hashlock is not responsible for the safety of any funds and is not in any way liable for the security of the project.

Technical Disclaimer

Smart contracts are deployed and executed on a blockchain platform. The platform, its programming language, and other software related to the smart contract can have their own vulnerabilities that can lead to attacks. Thus, the audit can't guarantee the explicit security of the audited smart contracts.

About Hashlock

Hashlock is an Australian-based company aiming to help facilitate the successful widespread adoption of distributed ledger technology. Our key services all have a focus on security, as well as projects that focus on streamlined adoption in the business sector.

Hashlock is excited to continue to grow its partnerships with developers and other web3-oriented companies to collaborate on secure innovation, helping businesses and decentralised entities alike.

Website: hashlock.com.au

Contact: info@hashlock.com.au



#hashlock.

